■ Heat Pump Feasibility & Simulation Software: MVP Specification

---

■ PART 1: DETAILED MVP SPECIFICATION

■ 1. MVP Overview

1.1 Purpose

Develop a **web-based MVP** for heat pump feasibility studies and basic simulation, targeting **residential/commercial buildings** (greenhouse/CEA support deferred to Phase 2). The MVP will:

- Enable **non-experts** (architects, contractors, homeowners) to **size and evaluate** heat pump systems.

- Provide **quick feasibility checks** with thermal and economic outputs.

- Serve as a **foundation** for future expansion (greenhouses, hybrid systems, advanced simulation).

1.2 Goals

|------|--------|--------|

1.3 Scope (Included in MVP)

■ **User Inputs**

- Building dimensions (area, volume, orientation)

- Envelope materials (walls, roof, floor, windows)

- Location (auto-climate data)

- Desired indoor temperature

- Heat pump type (air-source, ground-source)

■ **Core Calculations**

- Heat loss (BS EN 12831 or ASHRAE simplified)
- Heat pump sizing (capacity, COP/SCOP)
- Basic economic analysis (CAPEX, OPEX, simple payback)

■ **Outputs**

- Heat load profile (kW)
- Recommended heat pump size (kW)
- Estimated COP/SCOP
- Simple payback period (years)
- Carbon savings (vs. gas boiler)

■ **Database**

- Material properties (U-values, thermal conductivity)
- Climate data (temperature, humidity)
- Heat pump performance curves (generic models)

■ **UI/UX**

- Web-based (responsive)
- Guided input workflow
- Visual results (charts, heat loss map)

1.4 Scope (Excluded from MVP)

■ **Greenhouse/CEA mode** (Phase 2)

■ **Hybrid systems** (solar + heat pump + storage) (Phase 2)

■ **Dynamic simulation** (hourly/transient) (Phase 3)

■ **Advanced economics** (NPV, IRR, incentives) (Phase 2)

■ **Collaboration features** (Phase 3)

■ **BIM/Revit integration** (Phase 4)

■ **IoT monitoring** (Phase 4)

■ 2. User Stories & Requirements

2.1 User Personas

|---------|-------------|-------------|--------------|

2.2 Functional Requirements

FR-1: User Inputs*

|----|-------------|----------|-------|

FR-2: Calculations & Simulation*

|----|-------------|----------|-------|

FR-3: Economic Analysis*

|----|-------------|----------|-------|

FR-4: Outputs & Visualization*

|----|-------------|----------|-------|

FR-5: Database*

|----|-------------|----------|-------|

FR-6: System & UX*

|----|-------------|----------|-------|

■■ 3. System Architecture

3.1 High-Level Architecture

```
```mermaid graph TD A[Client: Next.js] -->|API Calls| B[Backend: FastAPI] B --> C[Simulation Engine: Python] B --> D[Database: PostgreSQL] D --> E[Material Properties] D --> F[Climate Data] D --> G[Heat Pump Models] C --> H[Calculation Modules] H --> H1[Heat Loss] H --> H2[Heat Pump Sizing] H --> H3[Economic Analysis] A --> I[Visualization: Plotly.js] B --> J[External APIs] J --> J1[OpenWeatherMap] J --> J2[Energy Price APIs] ```
```

graph TD

A[Client: Next.js] -->|API Calls| B[Backend: FastAPI]

B --> C[Simulation Engine: Python]
```

B --> D[Database: PostgreSQL]

D --> E[Material Properties]

D --> F[Climate Data]

D --> G[Heat Pump Models]

C --> H[Calculation Modules]

H --> H1[Heat Loss]

H --> H2[Heat Pump Sizing]

H --> H3[Economic Analysis]

A --> I[Visualization: Plotly.js]

B --> J[External APIs]

J --> J1[OpenWeatherMap]

J --> J2[Energy Price APIs]

```
``` ### **3.2 Component Diagram** ```mermaid
```

## ### **3.2 Component Diagram**

```
```mermaid graph TD A[Client: Next.js] -->|API Calls| B[Backend: FastAPI] B --> C[Simulation Engine: Python] B --> D[Database: PostgreSQL] D --> E[Material Properties] D --> F[Climate Data] D --> G[Heat Pump Models] C --> H[Calculation Modules] H --> H1[Heat Loss] H --> H2[Heat Pump Sizing] H --> H3[Economic Analysis] A --> I[Visualization: Plotly.js] B --> J[External APIs] J --> J1[OpenWeatherMap] J --> J2[Energy Price APIs] ```
```

classDiagram

class Frontend {

+Next.js (React)

+UI Components

+State Management (Zustand)

+Form Handling (React Hook Form)

}

class Backend {

+FastAPI

+RESTful Endpoints

+Authentication (JWT)

}

class SimulationEngine {

+HeatLossCalculator

+HeatPumpSizer

+EconomicAnalyzer

+DataValidator

}

class Database {

+PostgreSQL

+MaterialTable

+ClimateTable

+HeatPumpTable

}

class ExternalAPIs {

+OpenWeatherMap

+EnergyPriceAPI

}

Frontend --> Backend : HTTP Requests

Backend --> SimulationEngine : Calls

Backend --> Database : Queries

Backend --> ExternalAPIs : Fetches Data

SimulationEngine --> Database : Reads

```
``` ### **3.2 Component Diagram** ```mermaid
```

### **3.3 Data Flow**

1. **User Inputs** → Frontend validates → Sent to Backend (JSON).

2. **Backend** validates → Calls **Simulation Engine**.

3. **Simulation Engine**:

- Fetches **material/climate data** from Database.

- Runs **heat loss calculation**.

- Sizes **heat pump** and estimates **COP/SCOP**.

- Calculates **economics** (CAPEX, OPEX, payback).

4. **Backend** returns **results** (JSON) → Frontend renders.

5. **Frontend** displays **charts, tables, report**.

---

## **■■ 4. Data Model**

### **4.1 Database Schema (PostgreSQL)**
```

```sql -- Users (for future auth, MVP uses local storage) CREATE TABLE users (id SERIAL PRIMARY KEY, email VARCHAR(255) UNIQUE, password_hash VARCHAR(255), created_at TIMESTAMP DEFAULT NOW()); -- Projects (MVP: stored in localStorage; Phase 2: DB) CREATE TABLE projects (id SERIAL PRIMARY KEY, user_id INTEGER REFERENCES users(id), name VARCHAR(255), building_type VARCHAR(50), -- residential, commercial created_at TIMESTAMP DEFAULT NOW(), updated_at TIMESTAMP DEFAULT NOW()); -- Building Dimensions CREATE TABLE building_dimensions (id SERIAL PRIMARY KEY, project_id INTEGER REFERENCES projects(id), length FLOAT, -- meters width FLOAT, height FLOAT, floors INTEGER, orientation VARCHAR(20) -- N, S, E, W); -- Materials CREATE TABLE materials (id SERIAL PRIMARY KEY, name VARCHAR(100), description TEXT, thermal_conductivity FLOAT, -- W/m·K density FLOAT, -- kg/m³ specific_heat FLOAT, -- J/kg·K u_value FLOAT, -- W/m²·K (pre-calculated) type VARCHAR(50) -- wall, roof, floor, window); -- Building Surfaces (walls, roof, floor, windows) CREATE TABLE surfaces (id SERIAL PRIMARY KEY, project_id INTEGER REFERENCES projects(id), material_id INTEGER REFERENCES materials(id), surface_type VARCHAR(50), -- wall, roof, floor, window area FLOAT, -- m² thickness FLOAT -- m (for custom materials)); -- Windows CREATE TABLE windows (id SERIAL PRIMARY KEY, project_id INTEGER REFERENCES projects(id), glazing_type VARCHAR(50), -- single, double, triple area FLOAT, -- m² u_value FLOAT, -- W/m²·K shading_factor FLOAT DEFAULT 1.0); -- Locations CREATE TABLE locations (id SERIAL PRIMARY KEY, city VARCHAR(100), country VARCHAR(100), latitude FLOAT, longitude FLOAT, climate_zone VARCHAR(50)); -- Climate Data (cached from API) CREATE TABLE climate_data (id SERIAL PRIMARY KEY, location_id INTEGER REFERENCES locations(id), month INTEGER, -- 1-12 avg_temp FLOAT, -- °C min_temp FLOAT, max_temp FLOAT, humidity FLOAT -- %); -- Heat Pump Models CREATE TABLE heat_pump_models (id SERIAL PRIMARY KEY, name VARCHAR(100), type VARCHAR(50), -- air-source, ground-source heating_capacity_min FLOAT, -- kW heating_capacity_max FLOAT, cop_at_7C FLOAT, -- COP at 7°C outdoor cop_at_35C FLOAT, -- COP at 35°C outdoor efficiency_curve TEXT -- JSON: {temp: cop, ...}); -- Economic Data CREATE TABLE economic_data (id SERIAL PRIMARY KEY, country VARCHAR(100), electricity_price FLOAT, -- $/kWh gas_price FLOAT, -- $/kWh heat_pump_install_cost FLOAT, -- $/kW carbon_intensity_electricity FLOAT, -- g CO■/kWh carbon_intensity_gas FLOAT -- g CO■/kWh); -- Project Results (cached for performance) CREATE TABLE project_results (id SERIAL PRIMARY KEY, project_id INTEGER REFERENCES projects(id), heat_load FLOAT, -- kW annual_energy_demand FLOAT, -- kWh recommended_hp_capacity FLOAT, -- kW cop FLOAT, scop FLOAT, capex FLOAT, -- $ opex_annual FLOAT, -- $/year payback_years FLOAT, carbon_savings FLOAT, -- kg CO■/year created_at TIMESTAMP DEFAULT NOW()); ```

-- Users (for future auth, MVP uses local storage)

CREATE TABLE users (

id SERIAL PRIMARY KEY,

email VARCHAR(255) UNIQUE,

password_hash VARCHAR(255),

created_at TIMESTAMP DEFAULT NOW()

);

-- Projects (MVP: stored in localStorage; Phase 2: DB)

CREATE TABLE projects (

id SERIAL PRIMARY KEY,

user_id INTEGER REFERENCES users(id),

name VARCHAR(255),

building_type VARCHAR(50), -- residential, commercial

created_at TIMESTAMP DEFAULT NOW(),

updated_at TIMESTAMP DEFAULT NOW()

);

-- Building Dimensions
```

```
CREATE TABLE building_dimensions (
id SERIAL PRIMARY KEY,
project_id INTEGER REFERENCES projects(id),
length FLOAT, -- meters
width FLOAT,
height FLOAT,
floors INTEGER,
orientation VARCHAR(20) -- N, S, E, W
);

-- Materials
CREATE TABLE materials (
id SERIAL PRIMARY KEY,
name VARCHAR(100),
description TEXT,
thermal_conductivity FLOAT, -- W/m·K
density FLOAT, -- kg/m³
specific_heat FLOAT, -- J/kg·K
u_value FLOAT, -- W/m²·K (pre-calculated)
type VARCHAR(50) -- wall, roof, floor, window
);

-- Building Surfaces (walls, roof, floor, windows)
CREATE TABLE surfaces (
id SERIAL PRIMARY KEY,
project_id INTEGER REFERENCES projects(id),
material_id INTEGER REFERENCES materials(id),
surface_type VARCHAR(50), -- wall, roof, floor, window
area FLOAT, -- m²
thickness FLOAT -- m (for custom materials)
);

-- Windows
CREATE TABLE windows (
id SERIAL PRIMARY KEY,
project_id INTEGER REFERENCES projects(id),
glazing_type VARCHAR(50), -- single, double, triple
area FLOAT, -- m²
```

```
u_value FLOAT, -- W/m²·K
shading_factor FLOAT DEFAULT 1.0
);

-- Locations
CREATE TABLE locations (
id SERIAL PRIMARY KEY,
city VARCHAR(100),
country VARCHAR(100),
latitude FLOAT,
longitude FLOAT,
climate_zone VARCHAR(50)
);

-- Climate Data (cached from API)
CREATE TABLE climate_data (
id SERIAL PRIMARY KEY,
location_id INTEGER REFERENCES locations(id),
month INTEGER, -- 1-12
avg_temp FLOAT, -- °C
min_temp FLOAT,
max_temp FLOAT,
humidity FLOAT -- %
);

-- Heat Pump Models
CREATE TABLE heat_pump_models (
id SERIAL PRIMARY KEY,
name VARCHAR(100),
type VARCHAR(50), -- air-source, ground-source
heating_capacity_min FLOAT, -- kW
heating_capacity_max FLOAT,
cop_at_7C FLOAT, -- COP at 7°C outdoor
cop_at_35C FLOAT, -- COP at 35°C outdoor
efficiency_curve TEXT -- JSON: {temp: cop, ...}
);

-- Economic Data
```

CREATE TABLE economic_data (

id SERIAL PRIMARY KEY,

country VARCHAR(100),

electricity_price FLOAT, -- $/kWh

gas_price FLOAT, -- $/kWh

heat_pump_install_cost FLOAT, -- $/kW

carbon_intensity_electricity FLOAT, -- g CO■/kWh

carbon_intensity_gas FLOAT -- g CO■/kWh

);

-- Project Results (cached for performance)

CREATE TABLE project_results (

id SERIAL PRIMARY KEY,

project_id INTEGER REFERENCES projects(id),

heat_load FLOAT, -- kW

annual_energy_demand FLOAT, -- kWh

recommended_hp_capacity FLOAT, -- kW

cop FLOAT,

scop FLOAT,

capex FLOAT, -- $

opex_annual FLOAT, -- $/year

payback_years FLOAT,

carbon_savings FLOAT, -- kg CO■/year

created_at TIMESTAMP DEFAULT NOW()

);

```
``` ### **3.2 Component Diagram** ```mermaid
```

## ### **4.2 Sample Data (Materials)**

|----|------|------|------------------|----------------------------|-----------------|

## ### **4.3 Sample Data (Heat Pump Models)**

|----|------|------|------------------|------------------|-----------|-----------|

---
```

■■ 5. Core Algorithms

5.1 Heat Loss Calculation (BS EN 12831 Simplified)

Formula:

```
``` ### **3.2 Component Diagram** ```mermaid
```

Q = Σ (U × A × ΔT) + Q_ventilation + Q_infiltration

```
``` ### **3.2 Component Diagram** ```mermaid
```

Where:

- `Q` = Total heat loss (W)

- `U` = U-value of surface (W/m²·K)

- `A` = Area of surface (m²)

- `ΔT` = Temperature difference (indoor - outdoor) (°C)

- `Q_ventilation` = Heat loss from ventilation (W)

- `Q_infiltration` = Heat loss from infiltration (W)

Steps:

1. Calculate **transmission heat loss** for each surface (walls, roof, floor, windows).

2. Calculate **ventilation heat loss**:

```

Q_vent = 0.33 × n × V × ΔT

```

- `n` = Air changes per hour (default: 0.5 for residential)

- `V` = Volume of building (m³)

3. Sum all losses to get **total heat load (kW)**.

Example:

- Building: 100 m², 2.5m height → Volume = 250 m³

- Walls: 200 m², U=0.3 W/m²·K

- Roof: 100 m², U=0.2 W/m²·K

- Windows: 20 m², U=2.8 W/m²·K

- Outdoor temp: -5°C, Indoor temp: 20°C → ΔT = 25°C

- Ventilation: 0.5 ACH

```
``` ### **3.2 Component Diagram** ```mermaid
```

Q_walls = 0.3 × 200 × 25 = 1,500 W

Q_roof = 0.2 × 100 × 25 = 500 W

Q_windows = 2.8 × 20 × 25 = 1,400 W
```

Q_vent = 0.33 × 0.5 × 250 × 25 = 1,031 W

Total Q = 1,500 + 500 + 1,400 + 1,031 = 4,431 W ≈ 4.4 kW

```
``` ### **3.2 Component Diagram** ```mermaid
```

## ### **5.2 Heat Pump Sizing**

**Steps:**

1. **Peak Heat Load** = `Q_total` (from heat loss calculation).

2. **Safety Factor** = 1.15 (15% oversizing for cold climates).

3. **Recommended Capacity** = `Q_total × Safety Factor`.

4. **Select Heat Pump** = Smallest model with `Capacity ≥ Recommended Capacity`.

**Example:**

- Peak load = 4.4 kW

- Recommended capacity = 4.4 × 1.15 = **5.06 kW**

- Selected model: **Generic ASHP (Small, 5–10 kW)**

## ### **5.3 COP/SCOP Estimation**

**COP (Coefficient of Performance):**

- **Air-Source Heat Pump (ASHP):**

```
COP = 0.05 × T_outdoor + 2.5 (Simplified linear model)
```

- Example: At 7°C → COP = 0.05×7 + 2.5 = **2.85** (real-world: ~3.5)

- At -5°C → COP = 0.05×(-5) + 2.5 = **2.25**

- **Ground-Source Heat Pump (GSHP):**

- COP = **4.0** (constant, as ground temp is stable).

SCOP (Seasonal COP):

- Weighted average COP over the year (based on climate data).

- **Formula:**

```
SCOP = Σ (COP_i × H_i) / Σ H_i
```

- `COP_i` = COP at temperature `i`

- `H_i` = Heating degree hours at temperature `i`
```

Example (Simplified):

|-------|---------------|-----|--------------------------|-------------|

5.4 Economic Analysis

CAPEX:

- `Heat Pump Cost` = `$/kW × Capacity` (e.g., $1,500/kW)

- `Installation Cost` = `$/kW × Capacity` (e.g., $1,000/kW)

- **Total CAPEX** = Heat Pump Cost + Installation Cost

OPEX:

- `Annual Energy Demand` = `Heat Load / COP × Runtime` (kWh)

- `Annual Electricity Cost` = `Energy Demand × Electricity Price` ($/kWh)

Savings (vs. Gas Boiler):

- `Gas Boiler Efficiency` = 0.9 (90%)

- `Annual Gas Demand` = `Heat Load / 0.9 × Runtime` (kWh)

- `Annual Gas Cost` = `Gas Demand × Gas Price` ($/kWh)

- **Annual Savings** = Gas Cost - Electricity Cost

Simple Payback:

```
``` ### **3.2 Component Diagram** ```mermaid
```

Payback (years) = CAPEX / Annual Savings

```
``` ### **3.2 Component Diagram** ```mermaid
```

Carbon Savings:

```
``` ### **3.2 Component Diagram** ```mermaid
```

Carbon Savings (kg CO■/year) = (Gas Demand × Carbon Intensity Gas) - (Electricity Demand × Carbon Intensity Electricity)

```
``` ### **3.2 Component Diagram** ```mermaid
```

■■ 6. API Specification

6.1 Endpoints

|--------|----------|-------------|--------------|----------|

6.2 Request/Response Examples

***POST `/api/projects/{id}/calculate`**

Request:

```
```json { "building": { "length": 10.0, "width": 8.0, "height": 2.5, "floors": 1, "orientation": "S" }, "surfaces": [ { "type": "wall", "material_id": 1, "area": 50.0, "thickness": 0.22 }, { "type": "roof", "material_id": 5, "area": 80.0, "thickness": 0.2 } ], "windows": [ { "glazing_type": "double", "area": 15.0, "u_value": 2.8 } ], "location": { "city": "London", "country": "UK", "latitude": 51.5074, "longitude": -0.1278 }, "indoor_temp": 20.0, "heat_pump_type": "air-source", "existing_system": "gas-boiler" } ```
```

{

"building": {

"length": 10.0,

"width": 8.0,

"height": 2.5,

"floors": 1,

"orientation": "S"

},

"surfaces": [

{

"type": "wall",

"material_id": 1,

"area": 50.0,

"thickness": 0.22

},

{

"type": "roof",

"material_id": 5,

"area": 80.0,

"thickness": 0.2

}

],

"windows": [

{

"glazing_type": "double",

"area": 15.0,

"u_value": 2.8

}
```

],

"location": {

"city": "London",

"country": "UK",

"latitude": 51.5074,

"longitude": -0.1278

},

"indoor_temp": 20.0,

"heat_pump_type": "air-source",

"existing_system": "gas-boiler"

}

```
``` ### **3.2 Component Diagram** ```mermaid
```

**Response:**

```
```json { "building": { "length": 10.0, "width": 8.0, "height": 2.5, "floors": 1, "orientation": "S" }, "surfaces": [{ "type": "wall", "material_id": 1, "area": 50.0, "thickness": 0.22 }, { "type": "roof", "material_id": 5, "area": 80.0, "thickness": 0.2 }], "windows": [{ "glazing_type": "double", "area": 15.0, "u_value": 2.8 }], "location": { "city": "London", "country": "UK", "latitude": 51.5074, "longitude": -0.1278 }, "indoor_temp": 20.0, "heat_pump_type": "air-source", "existing_system": "gas-boiler" } ```
```

{

"heat_load": 4.43, // kW

"annual_energy_demand": 12000.0, // kWh

"recommended_hp": {

"model_id": 1,

"name": "Generic ASHP (Small)",

"capacity": 5.0, // kW

"cop": 3.5,

"scop": 4.0

},

"economics": {

"capex": 7500.0, // $

"opex_annual": 800.0, // $/year

"payback_years": 5.2,

"carbon_savings": 2500.0 // kg CO■/year

},

"heat_loss_breakdown": {

"walls": 1.5, // kW

"roof": 0.5, // kW

"windows": 1.4, // kW

"ventilation": 1.03 // kW

}

}

```
``` ### **3.2 Component Diagram** ```mermaid
```

---

# ## **■ 7. UI/UX Design**

## ### **7.1 Wireframes**

### #### ***1. Homepage / Landing***

```
``` ### **3.2 Component Diagram** ```mermaid
```

```
+-----------------------------------+
| +-------------------------+ |
| +-------------------------+ |
+-----------------------------------+
```

```
``` ### **3.2 Component Diagram** ```mermaid
```

### #### ***2. Input Workflow (Step 1: Building)***

```
``` ### **3.2 Component Diagram** ```mermaid
```

```
+-----------------------------------+
+-----------------------------------+
```

```
``` ### **3.2 Component Diagram** ```mermaid
```

### #### ***3. Input Workflow (Step 2: Materials)***

```
``` ### **3.2 Component Diagram** ```mermaid
```

```
+-----------------------------------+
| +-------------------------+ |
| +-------------------------+ |
+-----------------------------------+
```

```
``` ### **3.2 Component Diagram** ```mermaid
```

### #### ***4. Input Workflow (Step 3: Location & Settings)***

```
``` ### **3.2 Component Diagram** ```mermaid
```

+----------------------------------+

+----------------------------------+

```
``` ### **3.2 Component Diagram** ```mermaid
```

#### ***5. Results Dashboard****

```
``` ### **3.2 Component Diagram** ```mermaid
```

+----------------------------------+

| +----------------------------+ |

| +----------------------------+ |

| +----------------------------+ |

| +----------------------------+ |

| +----------------------------+ |

| +----------------------------+ |

+----------------------------------+

```
``` ### **3.2 Component Diagram** ```mermaid
```

#### ***6. Heat Loss Breakdown Chart****

```
```mermaid graph TD A[Client: Next.js] -->|API Calls| B[Backend: FastAPI] B --> C[Simulation Engine: Python] B --> D[Database: PostgreSQL] D --> E[Material Properties] D --> F[Climate Data] D --> G[Heat Pump Models] C --> H[Calculation Modules] H --> H1[Heat Loss] H --> H2[Heat Pump Sizing] H --> H3[Economic Analysis] A --> I[Visualization: Plotly.js] B --> J[External APIs] J --> J1[OpenWeatherMap] J --> J2[Energy Price APIs] ```
```

pie

title Heat Loss Breakdown

"Walls" : 34

"Roof" : 11

"Windows" : 32

"Ventilation" : 23

```
``` ### **3.2 Component Diagram** ```mermaid
```

#### ***7. COP vs. Temperature Chart****

```
```mermaid graph TD A[Client: Next.js] -->|API Calls| B[Backend: FastAPI] B --> C[Simulation Engine: Python] B --> D[Database: PostgreSQL] D --> E[Material Properties] D --> F[Climate Data] D --> G[Heat Pump Models] C --> H[Calculation Modules] H --> H1[Heat Loss] H --> H2[Heat Pump Sizing] H --> H3[Economic Analysis] A --> I[Visualization: Plotly.js] B --> J[External APIs] J --> J1[OpenWeatherMap] J --> J2[Energy Price APIs] ```
```

lineChart

title COP vs. Outdoor Temperature

xAxis Outdoor Temp (°C)

yAxis COP

line ASHP, 2.5, 2.85, 3.2, 3.5, 3.8

line GSHP, 4.0, 4.0, 4.0, 4.0, 4.0

```
``` ### **3.2 Component Diagram** ```mermaid
```

---

## ## **■■ 8. Technical Stack**

|----------|------------|---------------|

---

## ## **■ 9. Project Structure**

```
``` ### **3.2 Component Diagram** ```mermaid
```

heat-pump-sim/

■■■ frontend/ # Next.js App

■ ■■■ public/

■ ■■■ src/

■ ■ ■■■ components/ # Reusable UI components

■ ■ ■ ■■■ inputs/

■ ■ ■ ■■■ results/

■ ■ ■ ■■■ layout/

■ ■ ■■■ pages/ # Next.js pages

■ ■ ■ ■■■ index.tsx # Landing

■ ■ ■ ■■■ project/

■ ■ ■ ■ ■■■ [id].tsx

■ ■ ■ ■ ■■■ new.tsx

■ ■ ■ ■■■ api/ # API routes (if needed)

■ ■ ■■■ styles/ # Tailwind + custom CSS

■ ■ ■■■ hooks/ # Custom React hooks

■ ■ ■■■ utils/ # Helper functions

■ ■ ■■■ types/ # TypeScript types

■ ■ ■■■ store/ # Zustand state

■ ■■■ package.json

■ ■■■ next.config.js

■

■■■ backend/ # FastAPI

■ ■■■ app/

■ ■ ■■■ main.py # FastAPI app

■ ■ ■■■ api/ # API routes

■ ■ ■ ■■■ projects.py

■ ■ ■ ■■■ materials.py

■ ■ ■ ■■■ ...

■ ■ ■■■ core/ # Core logic

■ ■ ■ ■■■ heat_loss.py

■ ■ ■ ■■■ heat_pump.py

■ ■ ■ ■■■ economics.py

■ ■ ■■■ db/ # Database

■ ■ ■ ■■■ models.py # SQLAlchemy models

■ ■ ■ ■■■ session.py # DB session

■ ■ ■■■ schemas.py # Pydantic models

■ ■■■ tests/ # pytest tests

■ ■■■ requirements.txt

■ ■■■ Dockerfile

■

■■■ data/ # Static data (seed DB)

■ ■■■ materials.json

■ ■■■ climate.json

■ ■■■ heat_pumps.json

■

■■■ docs/ # Documentation

■ ■■■ API.md

■ ■■■ USER_GUIDE.md

■

■■■ README.md

```
``` ### **3.2 Component Diagram** ```mermaid
```

---

## **■ 10. Testing Strategy**
```

10.1 Unit Tests (Backend)

|--------|------------|----------|

Example Test (pytest):

```
```python # tests/test_heat_loss.py from app.core.heat_loss import calculate_heat_loss def test_heat_loss_walls(): surfaces = [{"u_value": 0.3, "area": 200, "delta_t": 25}] ventilation = {"air_changes": 0.5, "volume": 250, "delta_t": 25} result = calculate_heat_loss(surfaces, ventilation) assert result["total"] == pytest.approx(1500 + 1031.25, rel=0.01) # 1500 (walls) + 1031.25 (vent) ```
```

tests/test_heat_loss.py

from app.core.heat_loss import calculate_heat_loss

def test_heat_loss_walls():

surfaces = [{"u_value": 0.3, "area": 200, "delta_t": 25}]

ventilation = {"air_changes": 0.5, "volume": 250, "delta_t": 25}

result = calculate_heat_loss(surfaces, ventilation)

assert result["total"] == pytest.approx(1500 + 1031.25, rel=0.01) # 1500 (walls) + 1031.25 (vent)

```
``` ### **3.2 Component Diagram** ```mermaid
```

10.2 Integration Tests

|----------|------|----------|

10.3 Frontend Tests

|----------|------|------|

10.4 Validation Strategy

1. **Benchmark Against EnergyPlus/HPDM**

- Compare results for **standard test cases** (e.g., reference buildings).

- Target: **±5% accuracy**.

2. **User Testing**

- Recruit **5–10 HVAC contractors** for feedback.

- Track **time-to-completion** and **error rates**.

3. **Edge Cases**

- Extremely cold climates (-20°C).

- Highly insulated buildings (Passivhaus).

- Large commercial buildings (10,000 m²).

■ 11. Deployment Plan

11.1 Environment Setup

|----------|----------|---------|----------|

11.2 CI/CD Pipeline (GitHub Actions)

```
```yaml # .github/workflows/test-and-deploy.yml name: Test and Deploy on: push: branches: [main, develop] pull_request: branches: [main] jobs: test-backend: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-python@v4 with: python-version: "3.10" - run: pip install -r backend/requirements.txt - run: pytest backend/tests/ test-frontend: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-node@v3 with: node-version: 18 - run: npm install - run: npm test working-directory: frontend deploy-staging: needs: [test-backend, test-frontend] if: github.ref == 'refs/heads/develop' runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - run: vercel --token ${{ secrets.VERCEL_TOKEN }} --env staging working-directory: frontend - run: railway up --detach working-directory: backend env: RAILWAY_TOKEN: ${{ secrets.RAILWAY_TOKEN }} deploy-production: needs: [test-backend, test-frontend] if: github.ref == 'refs/heads/main' runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - run: vercel --token ${{ secrets.VERCEL_TOKEN }} --prod working-directory: frontend - run: railway up --detach --environment production working-directory: backend env: RAILWAY_TOKEN: ${{ secrets.RAILWAY_TOKEN }} ```
```

.github/workflows/test-and-deploy.yml

name: Test and Deploy

on:

push:

branches: [main, develop]

pull_request:

branches: [main]

jobs:

test-backend:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
```

```
- uses: actions/setup-python@v4
with:
python-version: "3.10"
- run: pip install -r backend/requirements.txt
- run: pytest backend/tests/

test-frontend:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: actions/setup-node@v3
with:
node-version: 18
- run: npm install
- run: npm test
working-directory: frontend

deploy-staging:
needs: [test-backend, test-frontend]
if: github.ref == 'refs/heads/develop'
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- run: vercel --token ${{ secrets.VERCEL_TOKEN }} --env staging
working-directory: frontend
- run: railway up --detach
working-directory: backend
env:
RAILWAY_TOKEN: ${{ secrets.RAILWAY_TOKEN }}

deploy-production:
needs: [test-backend, test-frontend]
if: github.ref == 'refs/heads/main'
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- run: vercel --token ${{ secrets.VERCEL_TOKEN }} --prod
working-directory: frontend
```

- run: railway up --detach --environment production

working-directory: backend

env:

RAILWAY_TOKEN: ${{ secrets.RAILWAY_TOKEN }}

```
``` ### **3.2 Component Diagram** ```mermaid
```

### **11.3 Monitoring**

|------|---------|

---

## **■ 12. MVP Development Timeline**

|------|------|-------|--------|

**Total MVP Duration: 24 Weeks (6 Months)**

---

---

# **■ Appendix: Sources & References**

- **Market Data**: Grand View Research, Market Research Future
- **Existing Tools**: GeoT*SOL, Polysun, EnergyPlus, HPDM, TESPy
- **Standards**: BS EN 12831, ASHRAE 90.1
- **Greenhouse/CEA**: Appropedia, PMC/NCBI, ScienceDirect
```